

ViT

1. Introduction

논문이 다루는 분야

- Computer Vision의 image classification(image recognition) task에 NLP의 표준 구조인 Transformer를 적용하는 연구.
- NLP에서는 Transformer + 대규모 pre-training → fine-tuning 패러다임이 정착했고, 100B 이상 파라미터까지 scaling해도 성능 포화가 보이지 않음. 반면 vision에서는 여전히 CNN(ResNet 계열)이 지배적.
- 필요성: Transformer의 계산 효율성과 scalability를 vision에서도 활용할 수 있다면, NLP처럼 "모델과 데이터를 키우면 성능이 계속 오르는" 구조를 vision에서도 얻을 수 있음.

해당 task에서 기존 연구 한계점

- **CNN + attention 결합 방식:** self-attention을 CNN과 함께 쓰거나 CNN의 일부 component만 대체 → 전체 구조는 여전히 CNN에 의존.
- **convolution을 완전히 대체한 attention 모델** (local attention, sparse attention, axial attention 등): 이론적으로는 효율적이지만 특수한 attention 패턴을 쓰기 때문에 현대 하드웨어 가속기(GPU/TPU)에서 효율적으로 구현하기 어렵고, 결과적으로 대규모로 scaling되지 못함.
- **Naive pixel-level self-attention:** 픽셀 수에 대해 quadratic cost → 실제 해상도 이미지에 적용 불가.
- 그 결과 대규모 image recognition에서는 여전히 ResNet 계열이 SOTA.

논문의 contributions

1. 최소한의 수정으로 표준 **Transformer**를 이미지에 직접 적용: 이미지를 고정 크기 patch로 나누고, 각 patch를 NLP의 token처럼 취급하여 선형 임베딩 후 Transformer encoder에 입력 (ViT).
2. **"Large scale training trumps inductive bias"**: ViT는 CNN이 가진 locality, translation equivariance 같은 inductive bias가 부족해 ImageNet 규모에서는 ResNet보다 약간 낮지만, 14M~300M 규모 데이터로 pre-training하면 CNN SOTA를 넘어섬을 실험적으로 보임.
3. **계산 효율성:** SOTA CNN(BiT, Noisy Student) 대비 적은 pre-training 연산량(TPUv3-core-days)으로 동등 이상의 성능 달성.
4. 표준 Transformer 구조를 그대로 쓰기 때문에 NLP용으로 최적화된 구현을 거의 그대로 사용 가능.

2. Related Work

- **NLP Transformer:** Vaswani et al.(2017)의 Transformer와 BERT(denoising self-supervised), GPT(language modeling) 계열의 대규모 pre-training → fine-tuning 흐름을 소개하며, ViT가 따르고자 하는 방향으로 제시.
- **Vision에서의 self-attention 근사 기법들:** Introduction에서 언급한 "특수 attention 패턴" 연구들을 구체화함.
 - local neighborhood에만 attention 적용 (Parmar et al., Ramachandran et al. 등)
 - Sparse Transformer의 global attention 근사

- block 단위 / 축(axis) 단위 attention (axial attention)
- 공통 한계로 "하드웨어 가속기에서 효율적으로 구현하려면 복잡한 엔지니어링이 필요"하다고 정리 → Introduction의 문제 제기와 일관됨.
- **가장 유사한 연구 - Cordonnier et al.(2020):** 2×2 patch + full self-attention으로 ViT와 구조적으로 거의 같음. 차별점을 두 가지로 명시:
 1. 대규모 pre-training을 통해 vanilla Transformer가 SOTA CNN과 경쟁/능가함을 보인 점
 2. 2×2 patch는 저해상도 이미지에만 적용 가능하지만, ViT는 16×16 등 큰 patch로 중간 해상도 이미지까지 처리
- **CNN + self-attention 결합:** feature map augmentation, DETR(object detection), video, 멀티 모달 등 → CNN 의존 구조라는 점에서 ViT와 구분.
- **iGPT:** pixel 단위 Transformer를 생성 모델로 unsupervised 학습, ImageNet 최대 72% → ViT는 supervised, patch 기반으로 훨씬 높은 성능.
- **대규모 데이터 기반 image recognition:** BiT, Noisy Student, CNN의 데이터 규모별 scaling 연구 등. 같은 데이터셋(ImageNet-21k, JFT-300M)을 쓰되 ResNet 대신 Transformer를 학습한다는 점을 차별점으로 제시.
- 전반적으로 "기존 연구 = CNN 구조 유지 or 특수 attention" vs "ViT = 표준 Transformer + 대규모 데이터"라는 구도로 차별성을 명확히 표현함.

3. 제안 방법론

Main Idea

이미지를 patch 시퀀스로 보고, 표준 Transformer encoder에 그대로 넣는다.

1. Patch embedding

- 입력 이미지 $x \in \mathbb{R}^{(H \times W \times C)}$ 를 $P \times P$ 크기의 patch N 개로 분할 → $x_p \in \mathbb{R}^{(N \times (P^2 \cdot C))}$, $N = HW/P^2$
- N 이 곧 Transformer의 입력 시퀀스 길이가 됨 (예: 224×224 , $P=16 \rightarrow N=196$)
- 각 patch를 flatten 후 학습 가능한 linear projection $E \in \mathbb{R}^{((P^2 \cdot C) \times D)}$ 로 D 차원에 매핑

2. [class] token

- BERT처럼 학습 가능한 x_{class} 를 시퀀스 맨 앞에 추가. encoder 마지막 층의 이 token 출력(z_{L+1})에 LayerNorm을 적용한 것이 이미지 표현 y .
- classification head: pre-training 시 hidden layer 1개 MLP, fine-tuning 시 single linear layer.

3. Position embedding

- 학습 가능한 1D position embedding $E_{\text{pos}} \in \mathbb{R}^{((N+1) \times D)}$ 를 patch embedding에 더함.
- 2D-aware embedding, relative embedding도 실험했으나 유의미한 이득이 없어 가장 단순한 1D 사용.

4. Transformer encoder (L개 층)

- $z'_l = \text{MSA}(\text{LN}(z_{l-1})) + z_{l-1}$
- $z_l = \text{MLP}(\text{LN}(z'_l)) + z'_l$
- Pre-LN 구조, MLP는 2층 + GELU.

5. Fine-tuning & 고해상도 처리

- pre-trained head 제거 후 zero-initialized $D \times K$ linear layer 부착 (K = downstream class 수).
- 고해상도로 fine-tuning할 때 patch 크기는 유지 → 시퀀스 길이가 늘어남 → pre-trained position embedding을 원본 이미지 상 위치에 따라 **2D interpolation**.

6. **Hybrid 구조 (변형)**: raw patch 대신 CNN(ResNet) feature map에서 patch(1×1 가능)를 추출해 입력으로 사용.

모델 크기 (BERT 설정을 따름):

Model	Layers	D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

표기: ViT-L/16 = Large 모델 + 16×16 patch. patch가 작을수록 시퀀스가 길어져 연산량 증가.

Contribution

- **Introduction과의 일관성**: "fewest possible modifications"라는 주장대로, 이미지 특화 요소는 (1) patch 분할, (2) fine-tuning 시 position embedding 2D interpolation 두 곳뿐. 나머지는 원본 Transformer 그대로임을 Method에서 명확히 보여줌.
- **Inductive bias에 대한 명시적 분석**: CNN은 locality, 2D neighborhood 구조, translation equivariance가 모든 층에 내장된 반면, ViT는 MLP만 local/translation equivariant하고 self-attention은 global. position embedding도 초기에는 2D 정보가 없어 공간 관계를 전부 데이터에서 학습해야 함 → "소규모 데이터에서는 불리, 대규모에서는 유리"라는 Introduction 주장의 이론적 근거를 제공.
- **효율성 주장의 근거**: 특수 attention 없이 표준 dense attention을 쓰기 때문에 기존의 효율적인 Transformer 구현을 그대로 활용 가능. patch 단위라 시퀀스 길이가 $N=HW/P^2$ 로 줄어 quadratic cost 문제도 완화.
- **구현 가능성**: 수식(Eq. 1~4), 모델 설정(Table 1), 학습 하이퍼파라미터(Table 3, 4: Adam $\beta_1=0.9$, $\beta_2=0.999$, batch 4096, weight decay 0.1, 10k warmup, fine-tuning은 SGD momentum 0.9, batch 512, 해상도 384 등), head 교체 방식까지 상세히 기술되어 있고 코드/모델도 공개되어 재현 가능성이 높음. 다만 핵심 성능이 비공개 데이터셋 JFT-300M에 의존한다는 점은 재현성 측면의 제약.

4. 실험 및 결과

Dataset

- **Pre-training**
 - ImageNet (ILSVRC-2012): 1k classes, 1.3M images
 - ImageNet-21k: 21k classes, 14M images (공개)
 - JFT-300M: 18k classes, 303M images (Google 내부)
 - downstream test set과 중복 제거(de-duplication) 수행
- **Transfer / 평가**
 - ImageNet (original val labels, ReaL labels), CIFAR-10/100, Oxford-IIIT Pets, Oxford Flowers-102

- VTAB-1k (19 tasks, task당 1,000개 학습 샘플): Natural / Specialized(의료·위성) / Structured(기하학적 이해) 그룹
- (Appendix) ObjectNet
- **평가 지표:** fine-tuning accuracy (주), linear few-shot accuracy (regularized least-squares로 closed-form 계산, 빠른 평가용)

Baseline

- **BiT-L (ResNet152x4):** 대규모 ResNet 기반 supervised transfer learning. BatchNorm → GroupNorm, weight standardization 적용 ("ResNet (BiT)").
- **Noisy Student (EfficientNet-L2):** ImageNet + label 제거한 JFT-300M으로 semi-supervised 학습, 당시 ImageNet SOTA.
- **VTAB 비교:** VIVI (ImageNet+YouTube 공동학습 ResNet), S4L (supervised + semi-supervised).
- **Hybrid (R50 + ViT):** ResNet50 feature map을 ViT 입력으로 사용.
- 공정한 비교를 위해 ResNet도 Adam으로 pre-training (Appendix D.1에서 이 설정이 ResNet에도 더 좋을 있음을 확인).

결과

(1) SOTA 비교 (Table 2) - "적은 연산으로 SOTA" 검증

	ViT-H/14 (JFT)	ViT-L/16 (JFT)	ViT-L/16 (I21k)	BiT-L	Noisy Student
ImageNet	88.55	87.76	85.30	87.54	88.4/88.5
ImageNet Real	90.72	90.54	88.62	90.54	90.55
CIFAR-100	94.55	93.90	93.25	93.51	-
VTAB (19)	77.63	76.28	72.72	76.29	-
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

- ViT-L/16(JFT)는 같은 데이터로 학습한 BiT-L을 모든 task에서 능가하면서 연산량은 약 1/15 수준.
- ViT-H/14는 ImageNet, CIFAR-100, VTAB 등 어려운 task에서 추가 향상. 그래도 기존 SOTA보다 연산량이 훨씬 적음.
- 공개 데이터 ImageNet-21k로 학습한 ViT-L/16도 준수한 성능 (TPUv3 8코어로 약 30일).
- VTAB 그룹별로는 Natural, Structured에서 ViT-H/14가 우세, Specialized는 BiT와 비슷.

(2) Pre-training 데이터 요구량 (Figure 3, 4) - "inductive bias vs 데이터 규모" 검증

- ImageNet만으로 pre-training하면 ViT-L이 ViT-B보다도 낮고 BiT에 뒤짐. ImageNet-21k에서 비슷해지고, JFT-300M에서야 큰 모델의 이점이 온전히 드러나며 BiT를 추월.
- JFT 부분집합(9M/30M/90M/300M) 실험: 동일 하이퍼파라미터, 추가 regularization 없이 모델 고유 특성만 비교. 작은 데이터에서는 ViT가 더 overfit (예: ViT-B/32는 9M에서 ResNet50보다 훨씬 나쁘지만 90M 이상에서 역전). ResNet은 일찍 plateau, ViT는 데이터가 늘수록 계속 향상.

- → "convolutional inductive bias는 소규모 데이터에서 유용하지만 대규모 데이터에서는 데이터에서 직접 학습하는 것이 충분하고 오히려 유리하다"는 Introduction의 핵심 주장을 직접 뒷받침.

(3) Scaling study (Figure 5) - 연산 효율성 검증

- JFT-300M에서 pre-training 연산량 대비 transfer 성능 비교: ViT는 같은 성능을 내는 데 ResNet보다 약 2~4배 적은 연산 사용.
- Hybrid는 작은 연산 예산에서 ViT보다 약간 좋지만, 모델이 커지면 차이가 사라짐.
- ViT는 실험 범위 내에서 성능 포화가 보이지 않음 → 추가 scaling 여지.

(4) 내부 표현 분석 (Figure 6, 7)

- Patch embedding filter의 주성분이 patch 내부 미세 구조를 위한 그럴듯한 basis function 형태.
- Position embedding이 가까운 patch일수록 유사해지고 행/열 구조까지 학습 → 2D 구조를 스스로 학습하므로 hand-crafted 2D embedding이 이득이 없었던 이유를 설명.
- Attention distance: 낮은 층에서도 일부 head는 이미지 전체를 보고(global), 일부는 좁은 영역만 봄 (CNN 초기 conv와 유사). 층이 깊어질수록 전체적으로 attention distance 증가. 분류에 의미 있는 영역에 attention 집중.

(5) Self-supervision (preliminary)

- BERT식 masked patch prediction으로 ViT-B/16이 ImageNet 79.9% 달성 (scratch 대비 +2%, supervised pre-training 대비 -4%).

(6) Ablation (Appendix)

- Position embedding: 없음(0.614) vs 1D/2D/relative(약 0.640~0.643) → 유무 차이는 크지만 방식 간 차이는 거의 없음.
- [class] token vs GAP: learning rate만 맞추면 비슷한 성능.
- Transformer shape: depth scaling이 가장 효과적(16층 이후 diminishing return), width는 효과가 가장 작음, patch 크기 축소는 파라미터 증가 없이 꾸준히 향상.
- 실행 속도/메모리: 비슷한 ResNet과 추론 속도는 비슷하고, 코어당 최대 batch size는 대형 ViT가 훨씬 커서 메모리 효율이 좋음.

5. 결론(배운점)

연구의 의의

- CNN 없이 순수 Transformer만으로 image classification SOTA를 달성하며, vision에서 Transformer가 backbone이 될 수 있음을 보인 전환점. 이후 DeiT, Swin, MAE, CLIP의 image encoder, 멀티모달 LLM의 vision encoder 등으로 이어지는 흐름의 출발점.
- "이미지 = patch token 시퀀스"라는 추상화 덕분에 vision과 language를 같은 구조로 다룰 수 있게 되었고, 이는 이후 멀티모달 모델 설계의 기반이 됨.
- "inductive bias vs 데이터 규모"라는 trade-off를 명확한 실험(데이터 크기별 비교, JFT 부분집합 실험)으로 정리한 점이 방법론 자체만큼 가치 있음.

한계점

- 핵심 결과가 비공개 JFT-300M에 의존 → 외부에서 완전한 재현이 어렵고, 대규모 데이터/연산 자원이 없는 환경에서는 CNN보다 불리. (이 한계는 이후 DeiT가 강한 augmentation, distillation으로 ImageNet만으로 학습 가능하게 하며 보완.)
- Classification에만 검증했고, detection·segmentation 같은 dense prediction은 future work로 남김. 단일 해상도의 flat한 feature map이라 multi-scale 특징이 필요한 task에는 바로 적용하기 어려움 (이후 Swin 등 계층적 구조로 보완).
- Self-attention 연산량이 patch 수에 대해 quadratic이므로 작은 patch나 고해상도에서는 비용이 급격히 증가.
- Self-supervised pre-training은 preliminary 수준이며 supervised와 4% 격차.

좋았던 점

- "최소한의 수정"이라는 원칙이 매우 일관됨. 이미지 특화 설계를 patch 분할과 position embedding interpolation 두 곳으로 한정하고, 그 외 구성(1D position embedding, [class] token)도 ablation으로 "더 복잡한 대안이 이득이 없음"을 보여 단순함을 정당화함.
- 주장 → 근거 실험의 매핑이 명확함: 성능/효율 주장은 Table 2와 Figure 5, inductive bias 주장은 Figure 3·4, 내부 동작 이해는 Figure 7로 각각 검증.
- 이론 FLOPs뿐 아니라 실측 추론 속도, 코어당 최대 batch size까지 측정해 하드웨어 효율성 주장을 실제 수치로 뒷받침한 점.

아쉬웠던 점

- Pre-training 효율 비교에서 아키텍처 외 요인(학습 스케줄, optimizer, weight decay 등)의 영향을 저자들도 인정하고 있어, Table 2의 연산량 비교만으로는 순수 아키텍처 효율을 단정하기 어려움 (Section 4.4의 controlled study로 일부 보완).
- ImageNet 규모에서 강한 regularization/augmentation을 충분히 탐색하지 않아, "데이터가 적으면 ViT가 불리하다"는 결론이 학습 방법의 한계인지 구조의 한계인지 완전히 분리되지 않음.

배운 점

- 모델 설계에서 inductive bias는 데이터 효율을 높이는 대신 scaling 상한을 제한할 수 있으며, 충분한 데이터가 있으면 범용 구조가 더 멀리 갈 수 있다는 관점.
- 특수한 연산 패턴보다 하드웨어 친화적인 표준 연산(dense matmul)을 쓰는 것이 실제 scaling에서 결정적이라는 점. 이론적 효율성과 실제 가속기 효율성은 다를 수 있음 (Axial attention이 TPU에서 매우 느렸던 사례).
- Patch 크기가 곧 시퀀스 길이를 결정하고, 이것이 성능과 연산량 사이의 핵심 조절 변수라는 점. Transformer에서 토큰 수가 비용을 좌우한다는 점은 LLM 추론에서 시퀀스 길이와 KV cache 크기가 비용을 좌우하는 것과 같은 맥락으로 이해할 수 있음.

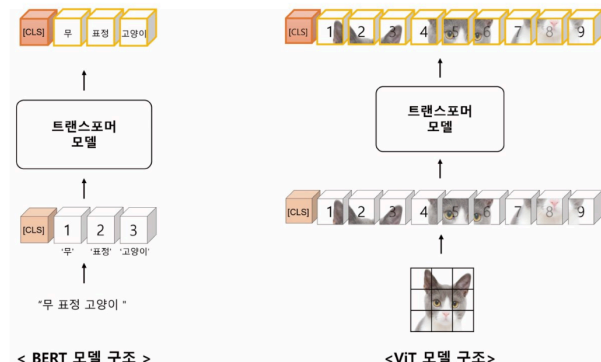
10. 비전 트랜스포머 — ViT(Vision Transformer)

[An Image is Worth 16 × 16 Words: Transformers for Image Recognition at Scale]

2020년 구글(Google)

이미지 인식을 위한 딥러닝 모델 – 이미지를 자연어 처리 방식처럼 분류해 보려는 시도에 의해 탄생함 (Transformer 구조 자체를 CV 분야에 적용한 첫 번째 연구)

• BERT(NLP) vs. ViT

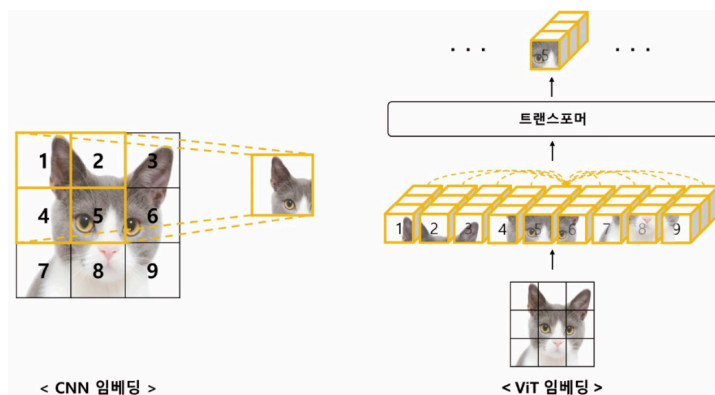


- **공통점:** 두 모델 다 트랜스포머 모델의 구조 그대로 사용
- **차이점:** 입력 데이터를 만드는 과정
 - **BERT:** 문장보다 작은 단위의 토큰들이 순차적으로 입력됨
 - **ViT:** 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨(입력 이미지 패치는 시퀀셜 배열—왼쪽에서 오른쪽, 위에서 아래—가정)

합성곱 모델 vs. ViT 모델

구분	합성곱 신경망(CNN)	ViT(Vision Transformer)
공통 목적	이미지 특징을 잘 표현하는 임베딩 생성	
학습 방식	이미지의 일부 영역만 선택해 학습 (e.g., 고양이 왼쪽 눈 부분) 작은 필터를 이미지 전체에 슬라이딩하며 국소 특징 추출 층이 깊어질수록 수용 영역 확대	이미지를 패치로 나눠 패치 간 상관관계를 학습
핵심 연산	합성곱 (좁은 수용 영역, RF)	Self-Attention (모든 패치 간 영향 고려)
전체 정보 표현	수용 영역이 좁아 많은 계층이 필요 깊은 층을 쌓아야 전체 정보 반영	어텐션 거리 덕분에 한 레이어로도 전체 정보 표현 가능
처리 단위	픽셀 단위	패치 단위 → 더 작은 모델로도 높은 성능 가능
입력 크기	(언급 X, 상대적으로 유연) Global pooling을 사용하면 다양한 크기 처리 가능	고정 → 크기가 다르면 resize , 위치 임베딩 보간 등 전처리 필요
위치 정보	공간적 위치 정보 고려	패치 간 상대적 위치만 고려 → 이미지 변환에 취약할 수 있음

e.g., 고양이 사진



- **CNN**
 - 고양이의 오른쪽 눈을 표현하는 데 1, 2, 4, 5만 학습되고 나머지(3, 6, 7, 8, 9) 이미지 패치는 학습에 관여 X
- **ViT**

- 셀프 어텐션 방법으로 모든 이미지 패치가 학습에 관여

ViT의 귀납적 편향(Inductive Bias)

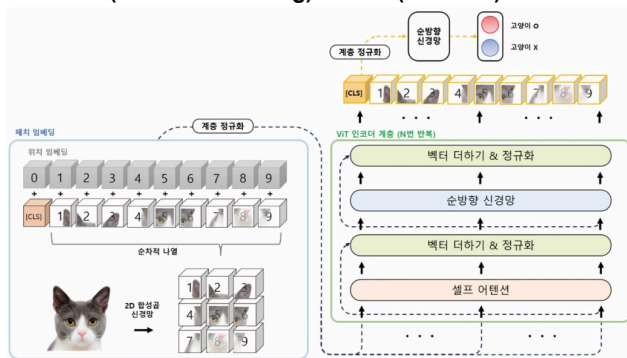
- 딥러닝 모델의 귀납적 편향: 일반화 성능 향상을 위한 모델의 가정(Assumption)
 - 해당 모델이 가지는 구조와 매개변수들이 데이터에 적합한 가정을 하고 있음
 - 이러한 가정이 올바르다면, 모델은 더 적은 데이터로 높은 일반화 성능 보일 수 있음
 - 너무 강한 귀납적 편향은 다른 유형의 데이터나 관계 표현에 어려움 초래할 수 있음

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 X

- e.g.1, 이미지 데이터—공간적 관계를 잘 표현함 → 지역적(Local) 편향을 가진 CNN 모델 많이 사용
 - **CNN**
 - 이미지 작은 조각으로 → 각 조각의 특징 추출 → 조합 → 전체 이미지 특징 추론
 - 지역적 특징 강조
- e.g.2, 시계열 데이터—시간적 관계를 잘 표현 → 시퀀셜(Sequential) 편향을 가진 RNN 모델 많이 사용
 - **RNN**
 - 이전 상태를 기억 + 현재 입력과 결합 → 다음 상태 예측
 - 순차적인 특징을 강조
 - 특정 관계 편향이 강할수록, 다른 다양한 관계를 표현하기 어려움 (중간에 정보가 희미해짐)
귀납적 편향이 약한 모델(가정을 덜 박아둔 모델. e.g., Transformer) 선호
- **ViT**
 - 입력 데이터의 다양한 Q, K, V의 임베딩 형태로 일반화된 관계를 학습 → 귀납적 편향 거의 X
 - 대용량 데이터에 대해 이미지 특징들의 관계를 잘 학습하는 모델로 알려져 있음

ViT 모델

패치 임베딩(Patch Embedding) + 인코더(Encoder) 계층

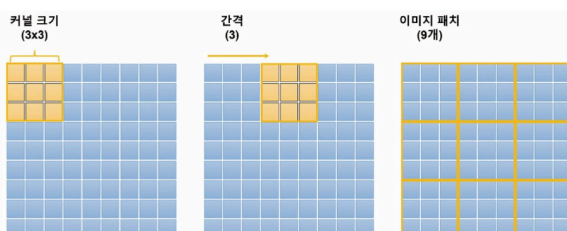


- 패치 임베딩: 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
- 인코더 계층: 각 패치와의 관계 학습
- 패치 임베딩과 인코더 계층을 통해 이미지의 특징을 추출하고 분류, 회귀와 같은 작업에 맞는 출력값으로 변환해 사용함

패치 임베딩(Patch Embedding)

입력 이미지를 작은 패치로 분할하는 과정

1. 전처리: 이미지 크기 맞추는 작업
e.g., 입력 이미지 크기 $640 \times 480 \rightarrow 224 \times 224$ (정방형 크기)
2. 패치 분할



- 합성곱 계층으로 전체 이미지를 패치 크기로 분할해 시퀀셜 배열 만들
- 커널 크기 = 패치 한 개의 크기
- 한 방향 패치 개수 = (이미지 크기 - 커널 크기) / 간격 + 1
 - e.g.1, 9×9 , 커널 3×3 , 간격 3 $\rightarrow (9 - 3) / 3 + 1 = 3 \rightarrow 3 \times 3 = 9$ 개 (겹침 없음, 위 그림)
 - e.g.2, 9×9 , 커널 3×3 , 간격 1 $\rightarrow (9 - 3) / 1 + 1 = 7 \rightarrow 7 \times 7 = 49$ 개 (겹침 있음)

3. [CLS] 토큰 추가

패치를 왼쪽 $\rightarrow$ 오른쪽, 위 $\rightarrow$ 아래 순서로 나열하고 맨 앞에 분류 토큰 **[CLS]** 를 붙임. 이미지 전체를 대표하는 벡터로 최종 예측에 사용

4. 위치 임베딩

패치의 위치 정보를 벡터로 만들어 각 패치 벡터에 더함 $\rightarrow$ 패치 간 위치 관계 학습

5. 계층 정규화

마지막에 적용하면 패치 임베딩 완성

- 원조 ViT에서 계층 정규화는 패치 임베딩 단계라기보다 이후 트랜스포머 인코더 블록 안에서 적용됨

- 실제 ViT는 커널 크기와 간격을 같게 뒤서(e.g., 16×16 , stride 16) 패치가 겹치지 않게 나누고, 합성곱의 출력 채널 수가 임베딩 차원이 됨

인코더 계층

패치 임베딩을 입력받아 패치 간 관계를 학습하고, [CLS] 벡터로 이미지를 분류하는 과정

1. 인코더 입력

패치 임베딩(패치 벡터 + [CLS] + 위치 임베딩)을 인코더 계층에 전달

2. 인코더 레이어 N개 반복

각 레이어에서 멀티 헤드 어텐션으로 Q, K, V 임베딩 간 관계를 학습(Transformer와 동일)

3. [CLS] 벡터 추출

마지막 레이어 출력 중 [CLS] 위치의 벡터만 꺼냄

- 이미지 전체를 잘 표현하는 특징 벡터로 간주
- 나머지 패치 출력(시퀀셜 산출물)은 분류에 쓰지 않음
- 분류, 검색 등 다양한 문제에 활용

4. 분류

[CLS] 벡터를 순방향 신경망(완전 연결 계층)에 연결해 예측

- e.g., 고양이인지 아닌지 이진 분류

- [CLS]는 실제 이미지에서 자른 패치가 아니라 학습 가능한 벡터를 시퀀스 앞에 붙인 것. 어텐션을 거치며 모든 패치의 정보를 모아 이미지 대표 벡터가 됨
- [CLS] 대신 전체 패치 출력을 평균 내서(global average pooling) 분류에 쓰는 방식도 있고, 성능 차이는 크지 않음

한계

이미지 패치가 왼쪽 $\rightarrow$ 오른쪽, 위 $\rightarrow$ 아래 방향으로 순차적으로 입력되어 2차원 구조의 이미지 특성을 온전히 반영하지 못함

(문제 해결) 물체의 크기나 해상도를 계층적으로 학습하는 모델 제안됨 — Swin Transformer, CvT 모델

- Swin Transformer: Local window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습시킴
- CvT: 기존 합성곱 연산 과정을 ViT에 적용한 모델. 저수준 특징(Low-level Feature)과 고수준 특징(High-level Feature)을 계층적으로 반영 가능
 - e.g., 사람 얼굴
 - 저수준 특징: 작은 단위(눈, 코, 입 등)
 - 고수준 특징: (눈, 코, 입을 포함한) 전체 얼굴
 - 어텐션 연산 과정에서 Q, K, V 중 K, V를 기존 특징 벡터보다 축소해 계산 복잡도를 감소시킴